

Designprinzipien in der modernen Software- entwicklung



Inhalt

1	Designprinzipien.....	2
1.1	Allgemeines.....	2
1.2	Übersicht.....	2
1.3	Grundlegende Designprinzipien	3
1.4	OOP Designprinzipien	3
1.5	Weitere Designprinzipien	4

1 Designprinzipien

"Software Architektur repräsentiert die frühen Entscheidungen, die ein System beeinflussen, während es noch leicht und kostengünstig ist, Änderungen vorzunehmen. Je besser diese Entscheidungen getroffen werden, desto erfolgreicher wird das System sein."

- Robert „Uncle Bob“ C. Martin

1.1 Allgemeines

Software-Designprinzipien sind grundlegende Richtlinien, die Anwendungsentwicklern helfen, Entscheidungen bei der Entwicklung von Softwaresystemen zu treffen. Sie stellen einen Rahmen dar, der Entwickler unterstützt effiziente, skalierbare und wartbare Software zu entwerfen, sowie Fehler zu minimieren.

Die Prinzipien sind als Richtlinien und Best Practices zu verstehen und können dazu beitragen, die Gesamtqualität von Software zu verbessern und erleichtern somit eine kostengünstige Entwicklung und Wartung.

1.2 Übersicht

Indem man die Designprinzipien befolgt oder zumindest berücksichtigt, kann man Quellcode sauberer, verständlicher und anpassungsfähiger gestalten. Die Systeme werden stabil, leicht erweiterbar, skalierbar und wartbar.

Das Wissen um Software-Design Prinzipien hilft Entwicklern qualitativ hochwertigen Code von schwer wartbarem oder schlecht designtem Code zu unterscheiden.

Im Laufe der Zeit haben sich unterschiedlichste Designprinzipien entwickelt. Wir wollen uns auf einige der wichtigsten Prinzipien beschränken.

1.3 Grundlegende Designprinzipien

- **Abstraktion** – Komplexität von Software wird reduziert und diese dadurch besser verständlich, leichter erweiterbar und wartbar.
- **Modularer Aufbau** – Softwaresysteme sollen aus einzelnen Modulen zusammengesetzt sein. Diese können separat entwickelt und getestet werden. Die Komplexität wird reduziert.
- **Strukturierung** – Softwaresysteme sollten über eine klare Struktur verfügen und in Schichten und Komponenten aufgeteilt sein. Entwickler können dadurch schnell und einfach Änderungen vornehmen.
- **Wiederverwendbarkeit** – Der Einsatz von wiederverwendbaren Modulen und Bibliotheken spart Zeit und Ressourcen. Diese sind getestet und stabil und müssen nicht jedes Mal neu entwickelt werden.
- **Separation of concerns** – Jede Softwarekomponente kann separat entwickelt werden. Abhängigkeiten werden reduziert, wenn sich jede Komponente auf eine spezifische Funktionalität konzentriert.

1.4 OOP Designprinzipien

Clean Code - Der Begriff „Clean Code“ und dessen Prinzipien, insbesondere die „SOLID“-Prinzipien wurden von Robert C. Martin eingeführt. Quellcode soll intuitiv verständlich sein. Ziel ist es Software effizient und effektiv zu produzieren, und den Quellcode so zu gestalten, dass dieser leicht

- lesbar
- änderbar
- erweiterbar
- wartbar

ist.

SOLID - Eine Menge von Designprinzipien wurden unter dem Akronym SOLID von Robert C. Martin zusammengefasst und gelten heutzutage als „State of the art“ moderner OOP. Jeder OOP-Entwickler sollte diese kennen.

S – Single Responsibility Principle

Das SRP verlangt, dass ein Modul mit Klassen, Funktionen, Variablen und Datenstrukturen nur für eine Sache verantwortlich sein sollte. Jede Klasse und auch jede Methode soll nur eine einzige fest definierte Aufgabe erfüllen und nur einen Grund zur Änderung haben. Diese

Aufgabe wird durch das Zusammenspiel aller Attribute und Methoden dieser Klasse erfüllt. Das Zusammenspiel der Attribute und Methoden dieser Klasse ist dadurch sehr eng.

O – Open Closed Principle

Das OCP thematisiert die Erweiterbarkeit von Modulen, Klassen und Methoden. Diese sollen offen für Erweiterungen und geschlossen für Modifikationen sein. Das bedeutet, dass das Verhalten einer Klasse durch Abstraktion, Vererbung oder Komposition erweitert werden kann, ohne den bereits bestehenden Code ändern zu müssen.

L – Liskov Substitution Principle

Das Ersetzbarkeitsprinzip LSP besagt, dass sich abgeleitete Subtypen wie ihre Basistypen verhalten müssen, wobei Subtypen die Funktionalitäten der Basistypen erweitern, aber nicht einschränken dürfen. Das Liskov-Prinzip ermöglicht die Polymorphie von Subtypen. Dies bedeutet, dass Objekte einer Basisklasse durch Objekte einer Unterklasse ersetzt werden können, ohne das korrekte Verhalten des Programms dadurch zu beeinträchtigen.

I – Interface Segregation Principle

Das ISP besagt, dass Interfaces lediglich zusammengehörige Elemente enthalten sollten. Ziel ist es, die Kopplung zwischen Komponenten “schlank” und übersichtlich zu gestalten. Wir verteilen Funktionalität auf viele schlanke Interfaces. Interfaces sollten klein und spezifisch sein und nicht zu viele Methoden enthalten. Das bedeutet, dass Klassen nur die Interfaces implementieren, die sie benötigen, und nicht unnötige Methoden implementieren müssen, die ein großes Interface eventuell vorschreibt.

D – Dependency Inversion Principle

Das DIP sieht vor, dass Module höherer Ebenen nicht von Modulen niedrigerer Ebenen abhängen. Module und Klassen sollen gegen Abstraktionen entwickelt werden und nicht von konkreten Typen und ihren Details abhängig sein. Das bedeutet, dass Klassen nicht direkt von anderen konkreten, ausdefinierten Klassen abhängen sollen, sondern von Interfaces oder abstrakten Klassen, die das gewünschte Verhalten definieren und vorschreiben. Wir erreichen also eine lose Kopplung zwischen den Softwarekomponenten.

1.5 Weitere Designprinzipien

DRY - Das Akronym steht für “Don’t repeat yourself” und bedeutet, dass Dopplungen im Code vermieden werden sollten. Redundanter Quellcode führt zu mehr Fehlern, mehr War-

tungsaufwand und weniger Flexibilität. Dazu sollte der Code in wiederverwendbare Methoden oder Klassen aufgeteilt werden, die an verschiedenen Stellen aufgerufen werden können.

KISS - Dies steht für “Keep it simple, stupid” und bedeutet, dass der Code so einfach wie möglich gehalten werden sollte. Komplexität führt zu mehr Fehlern, mehr Verwirrung und weniger Lesbarkeit. Um Komplexität zu reduzieren, sollte der Code in kleine und verständliche Einheiten aufgeteilt werden, die jeweils eine klare Absicht haben.

YAGNI - Dies steht für “You ain’t gonna need it” und bedeutet, dass keine unnötigen Funktionen oder Komponenten implementiert werden sollten. Oft neigen Entwickler dazu, vor auszudenken und Funktionen hinzuzufügen, die sie für zukünftige Anforderungen benötigen könnten.

Dies führt jedoch zu mehr Aufwand, mehr Komplexität und mehr technischen Schulden. Um dies zu vermeiden, sollte der Code nur das implementieren, was aktuell benötigt wird, und erst dann erweitert werden, wenn sich die Anforderungen ändern.